TOPCASINOSG · GUIDE FOR THE CONTENT TEAM

# Updating website content with AI

How an AI assistant can suggest changes to the website's pages and news, and how your team reviews and approves them, step by step.

### AI suggests

The assistant reads the site and writes improvements or new articles.

### You decide

Every suggestion waits in the dashboard until someone on your team approves it.

### Always undoable

Each page keeps its last 10 versions, so any change can be rolled back.

## What's inside


1 How it works

2 Creating a key for the AI

3 Connecting your AI assistant

4 Reviewing AI Changes

5 Undoing a change (Revisions)

6 What the AI can and can't do

7 Keeping it safe

8 Messages you may see

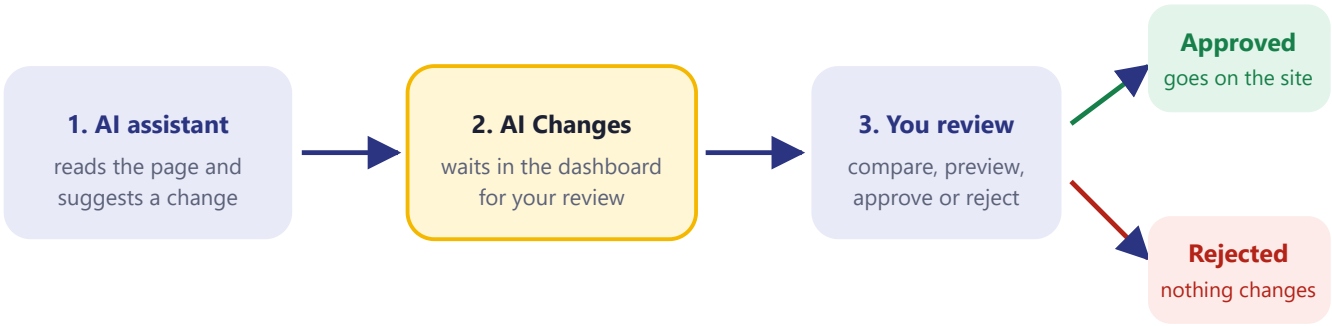
9 Questions & answers

A Appendix: technical reference

PART 1

# How it works

Think of the AI assistant as a helpful writer who can't press "Publish". It drafts the change, and your team decides whether it goes on the website.




## The three pieces

| PIECE | WHAT IT IS | WHERE |
|---|---|---|
| **API key** | A long secret code, like a password made for software. It lets the AI assistant talk to the website, but only for what you allow. | Dashboard › **Settings** › **API Keys** |
| **AI Changes** | The inbox where every suggestion from the AI waits. The menu shows how many are waiting. | Dashboard › **AI Changes** (under Content) |
| **Revisions** | The history of each page: the last 10 saved versions, ready to restore. | Any page › **Revisions** tab |

## What happens when you approve

- **A change to an existing page or article** goes on the website straight away (if that page is already published). The page's previous version is kept under Revisions.
- **A new page or article** is created as a **draft**. Nobody sees it until you open it and publish it the usual way.

> **The AI never publishes on its own.**
> Whatever the AI sends, nothing on the website changes until a person on your team clicks Approve.

PART 2

# Creating a key for the AI

Only a **Super Admin** or an **Admin** can create keys. It takes about a minute.

1. Sign in to the dashboard and open **Settings**, then the **API Keys** tab.
2. Click **+ Create Key**.
3. Give it a **name** that says who or what uses it, for example "*Content team AI assistant*". The name appears next to everything this key does, in AI Changes and in the Activity Log.
4. Tick what it may use:

| CHOICE | WHAT IT ALLOWS |
|---|---|
| **Pages** | Read the website's pages and suggest changes or new pages. |
| **News** | Read news articles and suggest changes or new articles. |
| **Media** | See the Media Library and upload images to it (for use in suggested content). |

   Only tick what's needed. A key that only writes news doesn't need Pages.

5. Click **Create Key**. The key is shown **once**. It starts with `tcs_`. Click **Copy** and keep it somewhere safe, such as your company's password manager.
6. Click **I've copied it**. From now on the list shows only the first few characters, never the whole key.

> **Treat the key like a password.**
> Don't paste it in emails, chat groups, documents or screenshots. Give it only to the AI tool that needs it. If you think it has been seen by someone else, revoke it (below) and create a new one; it takes a minute.

## Revoking a key

In **Settings** › **API Keys**, click **Revoke** next to the key. It stops working immediately. Suggestions it already sent stay in AI Changes, and you can still review them.

A key also stops working automatically if the admin who created it is disabled or removed. When a team member leaves, their keys go with them.

In the key list, **Key** shows the first characters (to tell keys apart), **Can use** the permissions you ticked, and **Last used** when the AI last used it ("Never" means it hasn't been connected yet).

PART 3

# Connecting your AI assistant

The website talks to AI tools through a **Content API**: a set of web addresses that software can call. Your AI tool needs two things: the address and the key.

| WHAT | VALUE |
| --- | --- |
| **Address** | `https://backapi.topcasinosg.com.sg/api/v1` |
| **Key** | The `tcs_…` key you copied, sent with every request as `Authorization: Bearer tcs_…` |

## Which AI tools can use it

Any AI tool or automation that can call web addresses ("make HTTP requests") with a key can use it. For example:

- **AI assistants with tools or actions**: Claude (with a connector or Claude Code), ChatGPT custom GPTs with Actions.
- **Automation platforms**: Make, Zapier, n8n (e.g. "every Monday, draft a news post from this feed").
- **Your own scripts**, if you have a developer.

**Coming next**

A ready-made connector, so Claude (and other assistants that support connectors) can be linked by pasting the address and key into its settings, with no setup by a developer. Until then, whoever sets up your AI tool can use the technical reference at the end of this guide.

**Tip: ask for small, clear changes**

"Update the introduction on the Thailand page for 2026" works better than "improve the website". Small suggestions are quicker to review.

PART 3, CONTINUED

## Instructions to give your AI assistant

Paste the text below into your AI assistant's instructions (in ChatGPT: the custom GPT's instructions; in Claude: the project instructions). Don't paste the key here: put it in the tool's own key setting.

```
You help update the TopCasinoSG website through its Content API at
https://backapi.topcasinosg.com.sg/api/v1 (send the API key as "Authorization: Bearer <key>").

How to work:
1. Start with GET /schema. It explains how pages are built and lists each block type's fields.
2. To change a page: GET /pages to find it, GET /pages/{id} to read it, change only the text
   that needs changing, and send the whole "sections" list back to POST /pages/{id}/changes.
   Keep every section you didn't change exactly as it was, in the same order.
3. For news: GET /news and GET /news/{id}; suggest changes with POST /news/{id}/changes,
   or a new article with POST /news (title, slug, excerpt, content).
4. Always include a short "note" saying what you changed and why.
5. Check first with "dryRun": true. Fix anything in "error" or "warnings", then send without dryRun.
6. Your changes are suggestions: a person reviews them in the dashboard before anything goes live.
   Tell the user what you submitted and share the "previewUrl" if there is one.
7. Use GET /changes to see whether earlier suggestions were approved or rejected, and read
   "reviewNote" for feedback.

Writing rules: match the tone and spelling already used on the site; be factual; never invent
bonus amounts, licences or other facts; keep the existing headings and structure unless asked;
no scripts or styles in HTML.
```

**Where the key goes**

Most tools have a separate, hidden place for keys or "credentials" (for example, the Authentication setting of a ChatGPT Action, or a connection in Make or Zapier). Put the `tcs_…` key there, choose "Bearer" if asked, and never inside the instructions text.

PART 4

# Reviewing AI Changes

Anyone with dashboard access (Super Admin, Admin or Editor) can review. The number next to `AI Changes` in the menu is how many are waiting.

1. Open `AI Changes` in the menu (under Content). The `Waiting for Review` tab lists every suggestion with its title, type, which key sent it, when, and the AI's note.
2. Click a suggestion. You'll see:
   - **Why**: the AI's own explanation of the change.
   - **What changes**: a table with the place on the page, the text **Now** (highlighted red) and the **Proposed** text (highlighted green). Only the parts that differ are listed.
3. For page changes, click `Preview on Site` to open the page as it would look with the change, in a new tab. Nothing is saved; the preview link works for one hour.
4. Optionally write a **Note to the tool**, e.g. "*Keep the bonus amounts as they are.*" The AI can read it and try again.
5. Decide:

| BUTTON | WHAT HAPPENS |
|---|---|
| `Approve and Apply` | The update is put on the site straight away. The previous version stays under the page's Revisions. |
| `Approve as Draft` | (new pages and articles) It's created as a draft. Click `Open Page` / `Open Article`, check it, then publish as usual. |
| `Reject` | Nothing changes. The suggestion moves to the Rejected tab with your note. |

**"Edited since"**

If someone edited the page after the AI read it, the suggestion shows **Edited since** and a yellow notice. The comparison is against the latest version. Approving would replace those edits where they overlap, so the button changes to `Approve Anyway`. If in doubt, reject it with a note asking the AI to start again from the latest version.

The other tabs keep the history: **Approved** and **Rejected** show who decided and when, with their note; **Withdrawn** lists suggestions the AI took back itself before review.

PART 5

## Undoing a change (Revisions)

Every time a page's content is saved, by a person in the editor or by approving an AI change, the website keeps that version. Each page keeps its **last 10**.

1. Open the page in **Pages** and click the **Revisions** tab.
2. Each version shows when it was saved, how (*Saved in the editor* or *Approved AI change*) and by whom. The top one is marked **Current**.
3. Click **Preview** to see an older version on the site without changing anything.
4. Click **Restore** to bring that version back. If the page is published, the website shows it straight away. The version you replaced stays in the list, so you can switch back.

Revisions cover the page content (its sections). The title, address and SEO fields aren't part of a revision.

PART 6

## What the AI can and can't do

| THE AI CAN | THE AI CAN'T |
|---|---|
| ✓ Read pages, news articles and the Media Library (as allowed by its key) | ✗ Publish, unpublish or schedule anything |
| ✓ Suggest text and layout changes to existing pages | ✗ Delete pages, articles or images |
| ✓ Suggest new pages and new news articles | ✗ Change a page's web address or move it in the page tree |
| ✓ Change a page's title and SEO title/description (as a suggestion) | ✗ Edit casinos, bonuses, payment methods, RTP, guides or the blacklist (for now) |
| ✓ Upload images to the Media Library | ✗ Change navigation, footer, sidebar, global sections or snippets |
| ✓ Withdraw its own suggestions and read your review notes | ✗ See or change users, settings, keys or the Activity Log |

Images uploaded by the AI go into the Media Library straight away (like any upload), but they only appear on the website once an approved change uses them.

PART 7

## Keeping it safe

- **Nothing goes live without a person.** Every page and news suggestion needs approval.
- **Everything is recorded.** The Activity Log shows each suggestion, upload and review, with the key's name, e.g. "*Jane Tan (API key: Content team AI assistant)*".
- **The website checks every suggestion** with the same rules as the page editor. A broken layout is refused before it reaches you.
- **Speed limit.** Each key can make up to 120 requests a minute, so a misbehaving tool can't flood the website.
- **Keys are stored safely.** The website keeps only a scrambled fingerprint of each key, never the key itself. That's why it's shown only once.
- **Mistakes are reversible.** Use Revisions to restore any of the last 10 versions of a page.

PART 8

## Messages you may see

When the AI tool gets an error, it usually shows the website's message. Here's what each kind means.

| MESSAGE TYPE | WHAT IT MEANS | WHAT TO DO |
|---|---|---|
| **Not valid / revoked** (401) | The key is wrong, was revoked, or its admin is no longer active. | Check the key was copied fully; create a new key if needed. |
| **This key can't use…** (403) | The key doesn't have that permission (e.g. Pages). | Create a key with the right boxes ticked. |
| **Invalid change** (400) | The suggestion breaks a rule, e.g. a component outside a Container. The message says exactly where. | The AI should fix it and send again. It usually does this by itself. |
| **Not found** (404) | The page, article or suggestion doesn't exist (or was deleted). | Ask the AI to list pages or articles again. |
| **Already reviewed / edited since** (409) | Someone already approved or rejected it, or the content changed in the meantime. | Refresh AI Changes and check the latest state. |
| **Too many requests** (429) | The key went over 120 requests a minute. | Wait a minute; the tool can then carry on. |
| **Warning: field may not show** | The AI used a field name that no section of that kind uses, often a typo. | The AI should use one of the field names the warning lists. |

PART 9

# Questions & answers

**Can the AI break the website?**

No. It can only suggest. Suggestions that would break a page's structure are refused automatically, and anything you approve can be undone with Revisions.

**Who can approve suggestions?**

Anyone who can edit content in the dashboard: Super Admins, Admins and Editors. Approving a suggestion is the same as making that edit yourself.

**Who can create or revoke keys?**

Super Admins and Admins, under Settings › API Keys.

**We lost the key. Can we see it again?**

No, it's shown only once. Revoke the old one and create a new key, then update it in your AI tool.

**Should each tool or person have its own key?**

Yes. Separate keys make it clear in AI Changes and the Activity Log who sent what, and you can revoke one without affecting the others.

**Two people edited the same page. Whose version wins?**

The website notices when a page changed after the AI read it and marks the suggestion "Edited since", so you can check before approving. Approving replaces the overlapping parts.

**Does the AI see unpublished drafts?**

Yes, it can read draft pages and articles (with the right permission), so it can help finish them. It still can't publish them.

**How long are suggestions kept?**

All suggestions stay in AI Changes, including approved and rejected ones, as a record.

**Can the AI update casinos, bonuses or other listings?**

Not yet. This first version covers pages, news and images. Other content types can be added in the same safe "suggest, then approve" way.

# Technical reference

Base address `https://backapi.topcasinosg.com.sg/api/v1`. Every request sends `Authorization: Bearer tcs_…` (or `X-API-Key: tcs_…`). Requests and responses are JSON. Responses look like `{"success": true, "data": {…}}` or `{"success": false, "error": {"message": "…"}}`.

| | ADDRESS | WHAT IT DOES | NEEDS |
|---|---|---|---|
| GET | `/me` | Which key this is, its permissions and the speed limit. | any key |
| GET | `/schema` | Page-building rules, field types, and every block type with the fields it uses (with examples). | any key |
| GET | `/pages` | All pages: id, path, title, status. | Pages |
| GET | `/pages/{id}` | One page with its full list of sections. | Pages |
| POST | `/pages/{id}/changes` | Suggest an update: any of `title`, `metaTitle`, `metaDescription`, `sections` (replaces the whole list), plus `note`. | Pages |
| POST | `/pages` | Suggest a new page: `title`, `slug`, optional `parentId`, `metaTitle`, `metaDescription`, `sections`, `note`. | Pages |
| GET | `/global-sections` | Reusable sections that can be placed on a page. | Pages |
| GET | `/news?page=1&pageSize=20` | News articles (without the full text). | News |
| GET | `/news/{id}` | One article with its full text. | News |
| POST | `/news/{id}/changes` | Suggest an update: any of `title`, `slug`, `excerpt`, `content`, `coverMediaId`, plus `note`. | News |
| POST | `/news` | Suggest a new article (`title` and `slug` required). | News |
| GET | `/media` | The Media Library (ids and image addresses). | Media |
| POST | `/media` | Upload a file (form field `file`); returns its id. | Media |
| GET | `/changes?status=pending` | This key's suggestions and their status and review notes. | any key |
| GET | `/changes/{id}` | One of this key's suggestions. | any key |
| POST | `/changes/{id}/withdraw` | Take back a suggestion that's still waiting. | any key |

# Page rules in short

- A page is an ordered list of sections. Level 0 (`"inContainer": false`) holds only a `container` or a `global_section`.
- Every other block is a component: it follows a container with `"inContainer": true` and belongs to the nearest container before it.
- Fields: `text` (plain text), `richtext` (HTML), `image` (`mediaId`), `button` (label in `textValue`, link in `urlValue`). Repeated items (e.g. FAQ questions) use `itemIndex` 0, 1, 2…
- Add `"dryRun": true` (or `?dryRun=1`) to check a suggestion without submitting it. Unknown JSON fields are refused, so typos don't go unnoticed.

### Example: suggesting a new news article

```
POST https://backapi.topcasinosg.com.sg/api/v1/news
Authorization: Bearer tcs_••••••••
Content-Type: application/json

{
  "note": "Weekly news round-up.",
  "title": "New Responsible Gambling Rules in Singapore",
  "slug": "new-responsible-gambling-rules-singapore",
  "excerpt": "What the new rules mean for players.",
  "content": "<p>From January, …</p>"
}
```

After approval, the article is created as a draft. Publish it from News in the dashboard.

APPENDIX · EXAMPLE

# Example: suggesting a page update

Read the page, change the text, and send the sections back with a note.

```
# 1. Read the page
GET https://backapi.topcasinosg.com.sg/api/v1/pages/12
Authorization: Bearer tcs_••••••••

# 2. Check the suggestion first (nothing is stored)
POST https://backapi.topcasinosg.com.sg/api/v1/pages/12/changes
Authorization: Bearer tcs_••••••••
Content-Type: application/json

{
  "note": "Updated the introduction for 2026.",
  "dryRun": true,
  "metaTitle": "Best Online Casinos in Thailand 2026",
  "sections": [
    { "blockType": "container", "inContainer": false, "fields": [] },
    { "blockType": "rich_text", "inContainer": true, "fields": [
      { "itemIndex": 0, "fieldKey": "body", "fieldType": "richtext",
        "textValue": "<p>Our updated guide to playing safely in 2026…</p>" }
    ] }
  ]
}

# Answer
{ "success": true, "data": { "valid": true, "warnings": [],
  "message": "This change is valid. Send it again without dryRun to submit it for approval." } }

# 3. Send the same request without "dryRun". Answer:
{ "success": true, "data": {
  "change": { "id": 41, "status": "pending", "label": "Thailand", … },
  "previewUrl": "https://topcasinosg.com.sg/preview/…",
  "message": "Submitted. It goes live only after an admin approves it in the dashboard (AI
Changes)." } }
```

**In a real request, send every section of the page**

The example is shortened. `sections` replaces the page's whole section list, so always start from what `GET /pages/{id}` returned and change only what's needed.